

## DAFI LABS PRIVATE LIMITED

DevOps Engineer Internship Program

---

# Week 1 Assignment Report

Git, GitHub, Vercel, DNS, Environment Variables & AI-Assisted DevOps

---

**Submitted by:** Omar Khaled Hussein

**Program:** Computer and Systems Engineering, Alexandria University

**Email:** [omar2004khaled@gmail.com](mailto:omar2004khaled@gmail.com)

**GitHub:** [github.com/omar2004khaled](https://github.com/omar2004khaled)

**Repository:** [github.com/omar2004khaled/Portfolio](https://github.com/omar2004khaled/Portfolio)

**Live URL:** [portfolio-chi-orcin-82.vercel.app](https://portfolio-chi-orcin-82.vercel.app)

**Date:** July 17, 2026

# Contents

---

|                                  |    |
|----------------------------------|----|
| Assignment Overview              | 2  |
| 1 Portfolio Website              | 3  |
| 2 GitHub Repository and Branches | 4  |
| 3 Vercel Deployment              | 5  |
| 4 DNS Research                   | 6  |
| 5 Environment Variables          | 7  |
| 6 DevOps Research                | 8  |
| 7 AI Learning Summary            | 9  |
| Conclusion                       | 10 |

## Assignment Overview

---

This report documents the completion of the Week 1 DevOps Engineer Internship assignment at Dafi Labs. The objective of the assignment was to build practical, hands-on confidence with the core tools of a modern web deployment workflow: Git and GitHub for version control, Vercel for continuous deployment, DNS for domain resolution, environment variables for configuration security, and AI-assisted tools for accelerated, self-directed learning.

The deliverable is a responsive, single-page developer portfolio built with plain HTML, CSS, and JavaScript (no build tooling or external frameworks), deployed live on Vercel and version-controlled on GitHub across three branches.

### Submission Checklist

- ✓ GitHub Repository Link
- ✓ Live Vercel URL
- ✓ Research Document (this report)
- ☐ Deployment Screenshots (*to be inserted — see Task 4 and Task 3 notes*)
- ✓ AI Conversation / Summary (Task 7)

# 1 Portfolio Website

## Task 1 — Create Portfolio

Marks: 30

A responsive, one-page developer portfolio was built using HTML5, CSS3, and vanilla JavaScript, with AI-assisted development (Cursor AI / ChatGPT) used to accelerate implementation while every feature was reviewed and understood before inclusion. The site runs entirely client-side, with no build step or package installation required.

### Key Features

- Fully responsive layout adapting to desktop, tablet, and mobile viewports
- Light / dark mode toggle with persisted user preference via `localStorage`
- Typewriter-style animated hero section cycling through role titles
- Sticky, scroll-aware navigation header with mobile hamburger menu
- Scroll-reveal animations powered by the Intersection Observer API
- Active-section highlighting in the navigation bar as the user scrolls
- Dedicated *Projects* section showcasing four key works
- *Contact* section with direct links (email, phone, location, GitHub, LinkedIn)
- A “View CV” action that opens the resume PDF in a new browser tab
- Back-to-top button for long-page navigation convenience

### Technology Stack

| Layer         | Technology                             |
|---------------|----------------------------------------|
| Markup        | HTML5 (semantic sectioning)            |
| Styling       | CSS3 (custom properties, Flexbox/Grid) |
| Interactivity | Vanilla JavaScript (ES6+)              |
| Icons         | Font Awesome 6.4.0 (CDN)               |
| Typography    | Google Fonts — Outfit & JetBrains Mono |

### Local Testing

The project was verified by opening `index.html` directly in a browser, as well as through the VS Code “Live Server” extension, confirming that all interactive features (theme toggle, typewriter effect, scroll reveals, mobile navigation) function correctly without a build pipeline.

## 2 GitHub Repository and Branches

### Task 2 — GitHub Repository

*Marks: 15*

The project was version-controlled with Git and pushed to a public GitHub repository.

**Repository:** <https://github.com/omar2004khaled/Portfolio>

### Branching Strategy

Three branches were created to reflect a standard environment-promotion workflow:

| Branch             | Purpose                                                                     |
|--------------------|-----------------------------------------------------------------------------|
| main               | Stable, production-ready codebase; source of truth for deployments          |
| development        | Active development branch for new features and fixes prior to review        |
| production/staging | Pre-production verification branch, mirroring what will be promoted to main |

Listing 1: Branch creation commands

```
git checkout -b development
git checkout -b production/staging
git push -u origin development
git push -u origin production/staging
```

### 3 Vercel Deployment

#### Task 3 — Deploy on Vercel

Marks: 20

The repository was connected to Vercel, which automatically detected the static site and deployed it without any custom build configuration.

**Live URL:** <https://portfolio-chi-orcin-82.vercel.app/>  
**Status:** HTTP 200 — verified reachable and rendering correctly

#### Deployment Notes

- No build command was required (static HTML/CSS/JS).
- No install command was required (no external dependencies to resolve).
- Vercel's GitHub integration triggers an automatic redeploy on every push to the connected branch, providing continuous deployment out of the box.

*Screenshot placeholder: Vercel deployment dashboard showing successful build and live status.*

## 4 DNS Research

### Task 4 — DNS Research

Marks: 20

A custom domain was not purchased for this assignment; the project is served through the free Vercel-provided subdomain. The DNS configuration required to attach a custom domain in the future was researched and is documented below for completeness.

### Required DNS Records for a Custom Domain on Vercel

| Type  | Host          | Value                             | Purpose                                              |
|-------|---------------|-----------------------------------|------------------------------------------------------|
| A     | @             | 76.76.21.21                       | Points the root/apex domain to Vercel's edge network |
| CNAME | www           | cname.vercel-dns.com              | Points the www subdomain to Vercel                   |
| TXT   | (as provided) | Vercel-issued verification string | Proves domain ownership during setup                 |

### Screenshots to Include Upon Domain Purchase

1. Vercel project domain settings page
2. DNS record configuration page from the domain registrar
3. Verified/active domain status confirmation in Vercel

*Screenshot placeholders: to be inserted here once a custom domain is connected.*

## 5 Environment Variables

### Task 5 — Environment Variables

Marks: –

A sample environment file, `.env.example`, was created to document the expected configuration keys without exposing any real secrets.

Listing 2: `.env.example` (illustrative structure)

```
# Example environment variables
API_BASE_URL=https://api.example.com
CONTACT_FORM_KEY=your_key_here
```

### Why `.env` Should Never Be Committed

- It may contain secret keys, tokens, and private credentials.
- Committing it can expose sensitive information publicly, especially in public repositories.
- It creates security risks if the repository is ever shared, forked, or made public later.

The real `.env` file is excluded from version control via `.gitignore`.

Listing 3: Relevant `.gitignore` entry

```
.env
```



## 6 DevOps Research

---

### Task 6 — DevOps Research

Marks: 20

#### GitHub

GitHub is a cloud-based platform for hosting Git repositories. It enables version control, collaborative development through pull requests and code review, issue tracking, and branch management — forming the backbone of modern software collaboration workflows.

#### Vercel

Vercel is a cloud platform purpose-built for deploying frontend and static web applications. It integrates directly with GitHub, automatically building and publishing a project on every push, and provides global CDN distribution, preview deployments, and free HTTPS out of the box.

#### DNS (Domain Name System)

DNS is the internet's naming system: it translates human-readable domain names into the IP addresses that computers use to locate servers, allowing browsers to find and connect to websites by name rather than numeric address.

#### Domains

A domain is the human-readable address a user types into a browser (e.g., `example.com`) to reach a website, rather than its underlying IP address.

#### .gitignore

The `.gitignore` file tells Git which files and directories to exclude from version control. It is typically used to omit local configuration, build artifacts, dependency folders (e.g., `node_modules`), and environment files containing secrets.

#### Build Command

The build command compiles and prepares a project for deployment (e.g., transpiling, bundling, minifying). This portfolio required no build command, as it is served directly as static HTML, CSS, and JavaScript.

#### Install Command

The install command fetches and installs a project's dependencies (typically via a package manager such as npm). No install command was needed here, since the project has no external package dependencies.

#### Environment Variables

Environment variables store configuration values — such as API keys, secrets, and environment-specific settings — outside of the source code, keeping sensitive data out of version control and allowing the same codebase to behave differently across development, staging, and production environments.

## 7 AI Learning Summary

### Task 7 — AI Learning

Marks: –

Using ChatGPT and Cursor AI as learning aids, the deployment of an application on a Ubuntu server using **Nginx** (as a reverse proxy) and **PM2** (as a Node.js process manager) was researched and understood step by step.

### Deployment Steps

1. Update the Ubuntu server's package index and installed packages.
2. Install Node.js and npm.
3. Install Nginx as the web server / reverse proxy.
4. Install PM2 globally to manage the Node.js process.
5. Upload the application code to the server.
6. Start the application using PM2 to keep it running persistently.
7. Configure Nginx to reverse-proxy incoming traffic to the PM2-managed app.
8. Verify the deployment by accessing the server's public address in a browser.

Listing 4: Representative command sequence

```
sudo apt update && sudo apt upgrade -y
curl -fsSL https://deb.nodesource.com/setup_lts.x | sudo -E bash -
sudo apt install -y nodejs nginx
sudo npm install -g pm2

# Deploy app
pm2 start app.js --name my-app
pm2 save
pm2 startup

# Configure Nginx reverse proxy, then:
sudo nginx -t
sudo systemctl reload nginx
```

### Reflection

This exercise clarified how source control (GitHub), managed deployment platforms (Vercel), and traditional server administration (Ubuntu, Nginx, PM2) represent complementary — rather than competing — approaches to shipping software, and how AI tools can accelerate learning a new stack while still requiring the developer to understand and verify each command before running it.

## Conclusion

All seven tasks of the Week 1 assignment were completed: a responsive portfolio was built and tested locally, version-controlled across three branches on GitHub, and deployed live on Vercel with a verified HTTP 200 response. DNS requirements for a future custom domain were researched and documented, environment variable hygiene was demonstrated through a sample `.env.example` and `.gitignore` entry, core DevOps concepts were explained in the researcher’s own words, and a full Ubuntu/Nginx/PM2 deployment workflow was learned with AI assistance and documented above.

| Criteria    | Marks |
|-------------|-------|
| Project     | 30    |
| GitHub      | 15    |
| Deployment  | 20    |
| Research    | 20    |
| Originality | 15    |
| Total       | 100   |

**Prepared by:** Omar Khaled Hussein

**Date:** July 17, 2026